



Technical Report

Architecture, Modules and Operation

Agent-based threat hunting and incident response for estates that do not have an EDR.

201	built-in detections	15	detection categories
148	API endpoints	22	database tables
37 / 11	collector modules (Win / Linux)	11	response actions
15	indicator feed sources	11	SIEM and alert destinations

Windows and Linux agents · FastAPI and SQLite console · Sigma, YARA and custom rule engines

01

What Douglas-042 is

Douglas-042 is an agent-based threat hunting and incident response platform. It sweeps a fleet of Windows and Linux hosts for signs of compromise, ranks what it finds, and lets an operator act on it — from a single console, with one deployment command per host.

It is built for the estates that need hunting most and can afford it least: a few dozen servers, no EDR budget, and an incident that started days ago. The usual alternatives are a commercial platform nobody has signed off on, or a folder of scripts somebody runs by hand and whose output lives in one person's terminal history.

Three design commitments

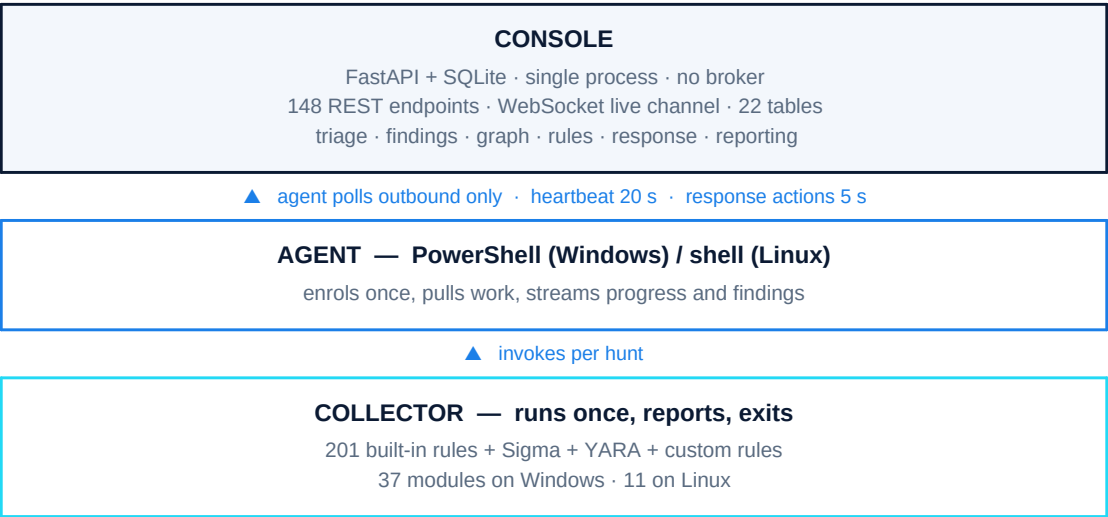
- **Nothing is installed on the host being investigated.** The collector is a single script that runs, reports and exits. The agent that schedules it is a scheduled task or a systemd unit, and it can be removed with one command.
- **The console never connects inward.** Agents poll it, so hosts behind NAT, a one-way firewall or a jump network still participate without inbound rules.
- **Silence is never mistaken for safety.** A module that failed and a module that found nothing produce the same empty result everywhere else; here they are separated, counted and reported.

The distinction that shapes the product

A sweep that could not look returns the same clean result as a sweep that looked and found nothing. Only one of those means the host is safe. Collection integrity rules, the capability check on Linux hosts, and the Logs & events screen all exist to keep those two outcomes apart.

02

Architecture



Component responsibilities

Component	Runs where	Lifetime	Responsibility
Console	One server or container	Long-lived	Stores findings, evaluates suppressions, scores risk, serves the UI and the API, distributes rules and indicators.
Agent	Each monitored host	Resident service	Enrols once, polls for hunts and response actions, streams progress and findings, uploads the evidence bundle.

Component	Runs where	Lifetime	Responsibility
Collector	Each monitored host	Runs and exits	Reads artifacts, applies the built-in, Sigma, YARA and custom rule sets, writes findings and a manifest.
Helpers	Linux hosts	Invoked by collector	Python evaluators for YARA file matching and Sigma log matching, kept separate so each can be tested on its own.

Why polling rather than a push channel

A push architecture needs the console to reach every host, which means inbound firewall rules on machines that are frequently the least trusted on the network. Polling inverts that: the host opens one outbound HTTPS connection and asks for work. The cost is latency, which is why response actions poll on a five-second interval of their own rather than waiting for the twenty-second heartbeat — containment should not queue behind a keep-alive.

Storage

One SQLite file. No message broker, no external database, no cluster. Schema migrations run at start-up and only ever add columns, indexes and tables — nothing is dropped or renamed, so an upgrade cannot lose an operator's tuning. The database holds 22 tables covering hosts, hunts, findings, cases, rules, feeds, reputation, response actions and the audit trail.

03

The console, screen by screen

The sidebar is organised by what an operator is doing rather than by which subsystem produced the data: **Operations** while a hunt is running, **Analysis** once results are in, **Setup** when tuning the platform, and **Diagnostics** when the results themselves look wrong. Screens marked with a role are hidden from accounts below it, rather than shown and then refused.

Operations

Screen	Role	What it is for
Cases	—	Groups related findings into an investigation with notes and an owner, so an incident survives being handed to the next shift.
Fleet	—	Every enrolled host with its posture, risk score and last sweep. Also where hunts are launched, and where Linux hosts report which detection capabilities they are missing.
Hunts	—	Running and completed sweeps. A live hunt shows the module feed as it happens — which module finished, how many rows it read, how many findings it produced and how long it took.
Response	responder	Eleven actions against a live host in three groups: look, act and contain. Read-only actions are visually separated from the six that change the host.

Analysis

Screen	Role	What it is for
Findings	—	Everything produced, filterable by severity, host, rule and status. Indicator matches are surfaced in a banner above the filters, since they are the one finding that needs no interpretation.
Triage	—	What still needs a decision. Confirmed keeps a finding in the risk score; false positive drops it for that host; a suppression hides the same finding on every future scan. Noisiest rules sit at the bottom because tuning those clears the queue fastest.
Frequency	—	How often each rule fires and across how many hosts. A rule firing everywhere usually describes how the estate is built rather than an intrusion.
Graph	—	Hosts and the external addresses they reached, ordered by risk. Confirmed indicator matches sort first, then reputation score; a known-good service sinks regardless of connection count.
ATT&CK	—	MITRE technique coverage for the latest sweep of each host, showing both what was detected and where the tool has no visibility.
Changes	—	What differs between two sweeps of the same host. New persistence between Tuesday and Thursday is a stronger signal than the same entry seen once.
Timeline	—	Findings and host events in time order, for reconstructing what happened and in which sequence.

Setup

Screen	Role	What it is for
DGL rules	—	The 201 built-in detections in 15 categories. Switchable one at a time or a whole family at once; click any rule to read what it checks, how it fires legitimately and what to do next.
My rules	—	Detections written locally against 11 artifact tables with 13 operators — in a form, or as text with validation that names the offending line. Bulk import and export in JSON, YAML or CSV.
YARA rules	admin	File-content rules, parsed at upload and evaluated on both platforms. Rules the parser cannot represent are rejected with a reason instead of being stored and left unable to fire.
Sigma rules	admin	Community log detections, downloaded and compiled per platform. Windows rules read the event log; Linux rules read auditd and the auth, cron and syslog text logs.
IOC feeds	—	Indicator sources merged into every hunt. Fifteen presets plus any URL returning indicator-shaped data. Watch-mode feeds are scanned for your own names and never sent to a host.
Threat intel	—	Reputation provider keys and the addresses already scored. Four providers, each answering a different question; verdicts are kept separate rather than averaged.
Integrations	admin	Eleven outbound destinations across SIEM, chat and case management, each with its own severity floor.
Users	admin	Console accounts and roles: viewer, responder, admin. Also the audit trail of who did what.
Schedules	—	Recurring sweeps, so hunting is a standing rhythm rather than something remembered during an incident.
Deploy agents	admin	The one-line deployment command per platform, the enrolment token, and an explanation of exactly what that command does.

Diagnostics

Screen	Role	What it is for
Logs & events	—	Whether the sweep could actually look. Every module on every hunt with its status, rows read, findings produced and duration; a per-module rollup; and, for admins, the console audit trail.

Why Logs & events is a screen and not a log file

Every other screen answers what is wrong. This one answers the question underneath it. A module that errored looked at nothing, and the host it errored on will report clean for exactly the things that module checks — which reads identically to a host that is genuinely clean.

04

The collector

The collector gathers what an analyst would gather by hand, then runs every rule set over it in one pass. Thirty-seven modules on Windows, eleven on Linux — the difference is not maturity but shape: Windows exposes many narrow sources (separate event log channels, the registry, WMI), while the Linux equivalents are fewer and broader.

Windows modules

Group	Modules
Scope and health	Scan scope · System information · Hotfixes and patch status · Event log health · Collection statistics
Live state	Users and groups · Process tree · Network connections · Services · Scheduled tasks · Named pipes · Drivers · SMB and shares · External endpoints
Persistence	Autoruns / ASEP · WMI persistence · Certificate stores
Event evidence	Process creation (4688) · PowerShell (4104/4103/400) · Logon activity · Account and policy changes · System (7045/7040/104) · RDP and WinRM · Persistence and defence channels · Sysmon · Kerberos (domain controllers)
Files and memory	File system scan · Webshell hunt · Exfiltration traces · User activity · In-memory code · Rootkit indicators · Raw forensic artifacts
Rule engines	Sigma rules · YARA scan · Custom rules

Linux modules

Module	What it reads
System information	Distribution, kernel, uptime, install age
Users and groups	Accounts, uid 0 duplicates, passwordless entries, sudoers, privileged group membership
Process tree	Running processes, deleted executables, command lines, working directories
Network	Connections, listening sockets, owning processes
Persistence	cron, systemd units, shell startup files, LD_PRELOAD, authorized_keys, sshd configuration
Kernel modules	Loaded modules, taint state
File system	setuid binaries, immutable attributes, recently written files
Web roots	Webshell patterns, WordPress, world-writable content
File content rules	YARA evaluation over candidate files
Sigma rules	auditd execution records and auth/cron/syslog logs
Logs and history	Authentication logs, shell history

Scope control

The collector does not read everything on the disk. File-content rules run against temporary directories, shared memory, web roots and home directories, restricted to files that are executable or were written inside the analysis window. A full sweep of a file server would take longer than the rest of the hunt and surface the same results.

05

Detection

Four rule sets run in the same pass. The built-in set is the baseline that works before anything is configured; the other three are what an operator adds.

Rule set	Source	Reads	Platforms
Built-in (DGL)	Ships with the collector	Every collected artifact	Windows, Linux
Sigma	Community repositories or upload	Event log (Win), auditd and text logs (Linux)	Windows, Linux
YARA	Uploaded by the operator	File content	Windows, Linux
Custom	Written in the console	11 artifact tables	Windows, Linux

The 15 detection categories

Category	What it covers
Collection integrity	Whether the sweep could see what it needed to — log retention, disabled channels, missing instrumentation.
Autoruns and startup	Anything that runs on its own at boot or logon.
Processes and network	What is running now and who it is talking to.
Services and tasks	Service and scheduled task configuration.
Persistence mechanisms	Things arranged to come back that are not a program: registry, WMI, cron, systemd, shell startup files, LD_PRELOAD.
Security posture	Whether the host's own defences are still switched on.
Drivers and kernel	Code running below the operating system.
Execution evidence	What ran earlier, reconstructed from logs.
Authentication and accounts	Who logged in, from where, and what changed.
Remote access	Remote execution and management: RDP, WinRM, SSH.
Sysmon, Kerberos and AD	The higher-fidelity evidence available when extra logging exists.
Files, webshells and exfiltration	What was written to disk, and what left.
User activity and trust	Traces a person leaves, and what the host trusts.
Rootkit and memory	Things hiding from the tools that would otherwise list them.
Indicator matches	A value from a loaded feed was found on the host.

Switching off is not suppressing

The two look similar and leave very different trails, which matters when somebody reads the result months later.

Action	Effect	Trail left
Switch off	The collector never runs the check. Nothing is produced.	None — there is no finding to review.

Action	Effect	Trail left
Suppress	The check runs; the finding is recorded and hidden from the queue.	The finding, marked as a standing decision with a reason and an author.

Switching off is enforced in two places: the collector is sent the disabled list and skips those checks, and the console drops findings from disabled rules on ingest so an older agent cannot reintroduce them. The list sent is the *disabled* one rather than the enabled one, so a failed fetch means every rule runs — failing toward more detection rather than less.

06

Indicators and reputation

These are frequently confused and do different jobs at different times. One applies what you already knew; the other explains what you found.

	Indicator feed	Reputation enrichment
Question	Is anything from this list on my hosts?	This host reached sixty addresses — which one first?
When	Applied during the hunt	After the hunt, in the background
Matched against	Process paths and hashes, network connections, DNS cache, file lists	External addresses the sweep already found
Result	A CRITICAL indicator-match finding — a confirmed identification	A score and a sentence of context per provider

Feed modes

An indicator feed's values are sent to every host. A watch feed's values never are — they describe victims rather than attackers, and loading a ransomware victim list as indicators would flag any machine that visited a listed company's website. Watch feeds are instead scanned in the console for the operator's own names. The separation is enforced by mode, not by trust in the source.

Reputation providers

Provider	Answers	Cost
AbuseIPDB	How many people reported this address, and for what	Free tier
VirusTotal	How many engines call it malicious	Free tier
ThreatFox	Whether it is known command-and-control, and for which malware family	Free
GreyNoise	Whether it scans the whole internet or came specifically for you	Paid — off by default

Verdicts are never blended

AbuseIPDB counts complaints, VirusTotal counts engines, ThreatFox identifies infrastructure and GreyNoise measures whether attention was targeted. An average of those four is a number none of them would defend. Each verdict is kept whole and the badge shows the worst one, named with the provider that gave it.

Protecting a free tier

- Results are cached for twelve hours, so a day of triage costs one lookup per address.
- Each run looks up at most forty addresses; the remainder is reported rather than silently dropped.
- A daily counter stops the console before the provider does, so a spent quota is visible rather than discovered through failures.
- Private and reserved addresses are never sent to a third party.
- A rejected key is recorded against the provider, never against an address — an authentication error says nothing about whether an address is malicious.

07

Incident response

A hunt says something is wrong. Response is the part where an operator does something about it, from the console, at an hour when the alternative is finding credentials for a machine nobody has logged into for a year.

Group	Action	Changes the host	What it does
Look	List running processes	No	Live process list with path, user and parent
Look	List network connections	No	Established and listening sockets with owning process
Look	Show a process and its ancestry	No	One process, its command line and the chain that started it
Look	Inspect a file	No	Size, timestamps, hash, signature, download origin
Look	List persistence entries	No	Autoruns, services and scheduled tasks, read live
Act	Kill a process	Yes	Confirms the name against the id before stopping it
Act	Quarantine a file	Yes	Moves rather than deletes, recording the original path
Act	Disable a local account	Yes	Disabled, not deleted; built-in accounts refused
Act	Stop a service	Yes	Stops and disables, so it does not return at reboot
Contain	Isolate the host	Yes	Blocks all traffic except the console, then proves the console is still reachable
Contain	Release the host	Yes	Removes only the rules isolation added

Safety properties

- **Isolation keeps the console reachable on purpose.** A host cut off from its own agent cannot be released remotely, and somebody would have to walk to it. The firewall rules keep one path open and the action verifies it before reporting success.
- **Targets that would break the host are refused, not warned about.** lsass.exe, systemd, pid 0 and pid 1, built-in accounts, and operating system paths. The moment this feature is used is the moment nobody is reading warnings.
- **Every mutating action requires a written reason.** Six months later the transcript is the only answer to why a production host changed.
- **The transcript is kept whether the action succeeded or not.** A failed containment attempt is precisely the thing that needs reading.
- **One action per host at a time,** enforced by a database constraint rather than a check two simultaneous requests can both pass.

08

What happens during a hunt

Stage	Console	Host
1. Queue	A hunt is created for one or more hosts and waits.	—
2. Pull	Answers the next heartbeat with the job, the indicator pool, the disabled-rule list and the rule bundles for that platform.	Fetches the collector if the hash changed.
3. Sweep	Receives module events and findings every ten seconds; findings appear in Findings, Triage and Frequency as they arrive.	Runs modules in order, appending findings to a stream as they are produced.
4. Upload	Reconciles the bundle against what already arrived live, applies standing suppressions, scores risk, forwards to integrations.	Uploads findings, timeline, graph snapshot, module statistics and collection errors.
5. Enrich	Scores the external addresses in the background, so triage order is correct before anyone opens the graph.	—

Live findings

A full sweep takes three to eight minutes. A critical finding produced in the second minute used to wait on the host until the bundle uploaded in the eighth. Findings now stream as they are found, and each carries its position in the sweep so the bundle can be reconciled against what already arrived — a retried upload cannot duplicate them, and a finding produced after the last progress tick is not lost.

Coverage reporting on Linux

Two hosts running the same distribution can differ on whether auditd is installed, whether it has rules loaded, and whether Python is present — and each decides whether a whole class of detection produces findings or silently produces nothing. The agent reports what it can run on every heartbeat, so installing auditd later is noticed without redeploying anything.

```
Detection capabilities on this host
auditd      : NOT RUNNING
              Nothing records what executes on this host, so
              execution rules cannot fire and a clean result
              only means the sweep could not look. Install it:
                apt install auditd
                systemctl enable --now auditd
file rules: available (python3 present)
```

09

Integrations and reporting

Findings that only exist in a console nobody has open are findings nobody acts on. Eleven outbound destinations in three groups, each with its own severity floor.

Group	Destinations	Written for
SIEM	Wazuh / generic JSON · Splunk HEC · Microsoft Sentinel · Elasticsearch · QRadar LEEF · ArcSight CEF · NDJSON	Indexing and later search. A low severity floor is appropriate.
Notify	Slack · Microsoft Teams · PagerDuty	A person reading it now, without opening anything. Keep the floor high or the channel gets muted.
Case	TheHive	Work somebody is expected to pick up and close.

Transport and delivery

- HTTP POST, syslog over UDP, syslog over TCP, or SMTP.
- TCP syslog is offered because UDP drops silently under load, and a feed that quietly loses findings is worse than one that fails loudly.
- Delivery is best effort and off the request path: a slow SIEM never fails a results upload that already succeeded.
- PagerDuty receives one event per batch with a deduplication key built from host and rule, so a noisy rule is one incident to work rather than forty pages.
- TheHive alerts carry the finding's own identity as their reference, so a re-send updates an existing alert instead of duplicating work somebody already closed.

Reporting

Each hunt produces a standalone HTML report containing the findings, the evidence, the analyst guidance for every rule that fired and the collection manifest. It opens in a browser with no console access required, which is what usually needs handing to a client or an auditor.

10

Security model

Roles

Role	Can
Viewer	Read every screen. Change nothing.
Responder	Launch hunts, triage findings, tune rules, manage feeds, run response actions.
Admin	Everything, plus accounts, integrations, reputation keys, rule downloads and agent deployment.

Platform hardening

- A password change is forced at first sign-in, and accounts lock after repeated failures.
- The session key is generated on first start with 0600 permissions. A fixed key shipped inside a package would be shared by everyone who downloaded it, and anyone holding it could mint a valid session cookie for somebody else's console.
- Agents authenticate with a per-host key issued at enrolment; the enrolment token itself is separate and revocable.
- Response actions are role-gated, audited, and refuse targets that would break the host.
- Rule exports neutralise spreadsheet formula injection, because a rule title is free text that may have arrived in a pack somebody else wrote.

- Uploaded rule files are size-capped and parsed with a safe loader.

Deployment footprint

The console needs Python 3.11 or newer and six dependencies, and runs comfortably on a 1 GB virtual machine. Windows hosts need PowerShell 5.1 and administrator rights, nothing else. Linux hosts need bash and curl; Python enables live progress and file-content rules, and auditd enables execution detections — and the console reports which are missing on which host rather than leaving it to be discovered.

APPENDIX

Deployment reference

Starting the console

```
pip install -r requirements.txt
uvicorn app.main:app --host 0.0.0.0 --port 8000

# or
docker compose up -d
```

Default sign-in

Username **admin**, password **douglas**. The console forces a password change at first sign-in, because a default baked into a package must not survive unnoticed. Set `DOUGLAS_CONSOLE_PASSWORD` in the environment file before the first start to choose your own; that file is only read while the database is still empty.

Enrolling a host

The console prints an enrolment token at start-up and shows the matching command on the Deploy screen. One line per platform, and the same line works on every host of that platform.

```
# Windows (elevated PowerShell)
iex (irm 'http://console:8000/api/v1/reports/deploy/script?token=TOKEN')

# Linux (as root)
curl -sSL 'http://console:8000/api/v1/reports/deploy/script\
?token=TOKEN&platform;=linux' | sudo bash
```

Six steps stream in the terminal: download the agent, enrol the machine, fetch the collector, install the service, confirm the process is alive, and check in. If enrolment fails, nothing is installed and the transcript says so.

A first hour that produces something

	Screen	Action	Result
1	IOC feeds	Add Feodo Tracker, press Refresh	Command-and-control detection, with no rule to write
2	Threat intel	Paste a free AbuseIPDB key, press Test	Triage order on the graph
3	Fleet	Select a host, launch a hunt	Findings, arriving live
4	Logs & events	Look for module errors	Whether the sweep could actually look
5	Triage	Work the queue, tune the noisiest rules	A list somebody still reads on the third pass
6	My rules	Load the worked examples and open one	The shape of a rule you can edit
7	Integrations	Slack webhook with the floor at HIGH	You hear about it without watching

Version 1.0. The Windows agent and collector are validated structurally but have not been executed in continuous integration; the first Windows deployment should be run with the terminal in view.